

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Казанский национальный исследовательский технический
университет им. А.Н. Туполева – КАИ»**

**Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности**

Лабораторная работа № 1

Дисциплина: «Аппаратные средства телекоммуникационных
систем»

Тема: «Основы параллельного программирования»

Работу выполнила
студент гр. 4404
Миронихина К.И.

Казань, 2026

Лабораторная работа №1

Задача 1.

Написать программу где каждый поток печатает свой идентификатор, количество потоков всего и строчку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

Код:

```
#include <iostream>
#include <omp.h>
using namespace std;

int main() {
    setlocale(LC_ALL, "Russian");
    int num_threads;
    cin >> num_threads;
    omp_set_num_threads(num_threads);
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int all = omp_get_num_threads();
        #pragma omp critical
        {
            cout << "Поток " << id << " из " << all << ". Hello World!" << endl;
        }
    }
    return 0;
}
```

Вывод:

```
8
Поток 0 из 8. Hello World!
Поток 1 из 8. Hello World!
Поток 3 из 8. Hello World!
Поток 4 из 8. Hello World!
Поток 5 из 8. Hello World!
Поток 6 из 8. Hello World!
Поток 2 из 8. Hello World!
Поток 7 из 8. Hello World!
```

Выводы могут быть различные, потому что потоки выполняются параллельно и независимо друг от друга. За то, какой именно поток будет выполняться в конкретный момент, отвечает планировщик ОС. Он решает, как распределить процессорное время между потоками, и этот процесс не контролируется напрямую программистом.

Задача 2.

Написать программу, в которой объявлен массив из 16000 элементов и инициализирован так, что значение элемента массива равно его порядковому номеру. Затем создайте результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива:

$$b[i] = (a[i-1] + a[i] + a[i+1])/3.0$$

Запустите программу с 8-ю процессами при различных типах распределения работ.

Код:

```
#include <iostream>
#include <omp.h>
#include <ctime>
#include <string>

int main() {
    setlocale(LC_ALL, "Russian");
    const int N = 16000;
    int a[N];
    double b[N];
    int num_thread;
    std::string schedule_type;
    std::cout << "Введите количество потоков: ";
    std::cin >> num_thread;
    std::cout << "Введите тип schedule: ";
    std::cin >> schedule_type;

    for (int i = 0; i < N; i++) {
        a[i] = i;
    }

    for (int i = 0; i < N; i++) {
        b[i] = 0.0;
    }

    omp_set_num_threads(num_thread);

    double start_time = omp_get_wtime();
    if (schedule_type == "static") {
        #pragma omp parallel for schedule(static)
        for (int i = 1; i < N - 1; i++) {
            b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
        }
    }
    else if (schedule_type == "dynamic") {
        #pragma omp parallel for schedule(dynamic)
        for (int i = 1; i < N - 1; i++) {
            b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
        }
    }
    else if (schedule_type == "guided") {
        #pragma omp parallel for schedule(guided)
        for (int i = 1; i < N - 1; i++) {
            b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
        }
    }
    else if (schedule_type == "runtime") {
        #pragma omp parallel for schedule(runtime)
        for (int i = 1; i < N - 1; i++) {
            b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
        }
    }

    double end_time = omp_get_wtime();
    std::cout << "Время выполнения: " << end_time - start_time
    << " секунд" << std::endl;

    std::cout << "Проверка первых 10 элементов:\n";
    for (int i = 0; i < 10; i++) {
        std::cout << "b[" << i << "] = " << b[i] << "\n";
    }

    return 0;
}
```

Выводы:

```
Введите количество потоков: 8
Введите тип schedule: static
Время выполнения: 0.0034082 секунд
Проверка первых 10 элементов:
b[0] = 0
b[1] = 1
b[2] = 2
b[3] = 3
b[4] = 4
b[5] = 5
b[6] = 6
b[7] = 7
b[8] = 8
b[9] = 9
```

При статическом распределении все итерации делятся поровну между потоками. Каждый поток получает свой фиксированный диапазон итераций. Размер блоков одинаковый

```
(Введите количество потоков: 8
aВведите тип schedule: dynamic
Время выполнения: 0.00395 секунд
Проверка первых 10 элементов:
(b[0] = 0
bb[1] = 1
b[2] = 2
b[3] = 3
sb[4] = 4
b[5] = 5
lb[6] = 6
sb[7] = 7
#b[8] = 8
fb[9] = 9
```

При динамическом распределении итерации разбиваются на блоки одинакового размера. Потоки по запросу берут следующий блок. Размер блока фиксированный

```
Введите количество потоков: 8
Введите тип schedule: guided
Время выполнения: 0.003053 секунд
Проверка первых 10 элементов:
b[0] = 0
b[1] = 1
b[2] = 2
b[3] = 3
b[4] = 4
b[5] = 5
b[6] = 6
b[7] = 7
b[8] = 8
b[9] = 9
```

Guided распределение похоже на динамическое. Отличие в том, что размер блока уменьшаются к концу.

Runtime – тип schedule выбирается не в коде, а через переменную окружения OMP_SCHEDULE.

Задача 3.

Модифицируйте задачу 1 так, чтобы потоки распечатывали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения.

```
✓ #include <iostream>
  [ #include <omp.h>
    using namespace std;

✓ int main() {
  |   setlocale(LC_ALL, "Russian");
  |   omp_set_num_threads(8);
  |   int all = omp_get_max_threads();
  |
  |   ////////////// 1 способ
  |
  |   #pragma omp parallel for ordered
  |   for (int i = 0; i < all; i++) {
  |       #pragma omp ordered
  |       {
  |           int id = all - 1 - i;
  |           cout << "Поток " << id << " из " << all << endl;
  |       }
  |   }
  }
```

Первый способ использует директиву `ordered` внутри параллельного цикла: благодаря `ordered` итерации выполняются строго по порядку ($i = 0, 1, 2, \dots$), а внутри каждой итерации мы вычисляем `id` как $all - 1 - i$, поэтому сначала печатается последний поток, потом предпоследний, и так до нулевого.

```
✓ ////////////// 2 способ
  |
  |   int ids[8];
  |   #pragma omp parallel
  |   {
  |       int id = omp_get_thread_num();
  |       ids[id] = id;
  |   }
  |
  |   for (int i = all - 1; i >= 0; i--) {
  |       cout << "Поток " << ids[i] << " из " << all << endl;
  |   }
  }
```

Каждый поток записывает свой номер в общий массив `ids` по своему индексу, а после завершения параллельного блока, проходя по этому массиву в обратном порядке, обычным последовательным циклом выводятся `id`.

```

////////// 3 способ

#pragma omp parallel
{
    int id = omp_get_thread_num();
    #pragma omp barrier
    for (int memb = all - 1; memb >= 0; memb--) {
        if (id == memb) {
            #pragma omp critical
            {
                cout << "Поток " << id << " из " << all << endl;
            }
        }
        #pragma omp barrier
    }
}

```

Третий способ использует барьеры и проверку `if (id == memb)`: все потоки выравниваются по барьеру, затем в цикле от `all-1` до `0` на каждой итерации только тот поток, чей `id` совпадает с текущим значением `memb`, заходит в критическую секцию и печатает свой номер, после чего все потоки снова синхронизируются через барьер перед следующей итерацией

```

////////// 4 способ

int maxid = all - 1;

#pragma omp parallel
{
    int id = omp_get_thread_num();
    #pragma omp barrier
    for (int memb = all - 1; memb >= 0; memb--) {
        #pragma omp critical
        {
            if (id == maxid) {
                cout << "Поток " << id << " из " << all << endl;
                maxid--;
            }
        }
        #pragma omp barrier
    }
}

return 0;
}

```

В четвертом способе используется общая переменная `maxid`, которая уменьшается после каждой печати: внутри критической секции каждый поток проверяет, равен ли его `id` текущему значению `maxid`, и если да — печатает и уменьшает `maxid`, а барьер внутри цикла заставляет всех ждать, пока нужный поток напечатает